

Named-Entity Recognition & Entity Resolution

Turning the Solanus Casey manuscripts into a knowledge graph's atoms

An explanatory packet for `stages/extract_entities.py` and
`stages/resolve_entities.py`

Solanus Project Pipeline — step_7

Abstract

This packet explains the two stages that convert handwriting into the typed, de-duplicated facts every later layer (the knowledge graph, temporal retrieval, GraphRAG) is built from. The first stage *finds* entities — it reads each letter and notebook entry and emits a list of *typed mentions*. The second stage *groups* them — it decides which of the many surface spellings point at the same real-world thing and mints one authority record per entity. We build intuition first (*mention* vs. *entity*; why blocking exists), then the mechanics (the two-pass NER, the rich nested schema, `normalize → block → match → cluster → canonicalize`), then the gated power-ups (embeddings, LLM adjudication, authority control), then the caveats. Throughout, the running example is a real corpus pattern: the many ways the notebooks spell *Grace Panyard*.

Contents

1	The one-sentence mental model	2
2	NER, Pass A: the free structured harvest	3
3	NER, Pass B: LLM extraction over the prose	3
3.1	Why a schema, not “return JSON”	4
3.2	GLiNER vs. the LLM: a two-tier extractor	4
3.3	Coreference → relation → event: the KG trinity	4
3.4	Cost discipline (nothing bills by accident)	5
4	Entity resolution: the “grouping,” step by step	5
4.1	Step 1 — Normalize: messy surface → stable keys	5
4.2	Step 2 — Block: cheap candidate generation (avoid $O(n^2)$)	5
4.3	Step 3 — Match: an ensemble of <i>interpretable</i> signals	6
4.4	Step 4 — Cluster: agreeing pairs → entities	6
4.5	Step 5 — Canonicalize: one authority record per entity	7
5	The three gated power-ups	7
6	Scale-up path: Splink	7
7	How to run	8
8	Caveats	9

TL;DR

- **Find, then group.** NER turns pages into *typed mentions*; resolution turns mentions into *entities*. Mentions are occurrences; entities are real-world things.
- **NER is two passes to save money.** Pass A *harvests* entities for **free** from fields `step_6` already separated; Pass B sends only the hard free text to **Gemini with a strict response_schema** (constrained decoding $\Rightarrow$ always-valid, nested JSON). Pass B is **gated**: it spends nothing until `--execute`.
- **The schema is rich and nested:** PERSON (+age/role/order, with CONDITIONS and RELATIONS hanging off the person), PLACE, ORG, DATE, CONDITION, FAVOR (+inline OUTCOME), OUTCOME, RELATION, ROLE__TITLE, RELIGIOUS__TERM.
- **Provenance on every mention:** `doc_id+rid+page+vertices+min_conf`, so an answer cites the exact polygon on the page.
- **Resolution = the classic ER pipeline:** `normalize  $\rightarrow$  block  $\rightarrow$  match  $\rightarrow$  cluster  $\rightarrow$  canonicalize  $\rightarrow$  (authority reconcile)`, running **offline at \$0** today; embeddings, LLM adjudication, and Wikidata/VIAF/GeoNames/Getty linking are wired but **gated**.
- **Conservative by design:** a wrong *merge* corrupts retrieval, so thresholds favour precision and grey-zone clusters go to David's review queue — never auto-merged — with a full audit log.

1 The one-sentence mental model

You can hold both stages in your head as a single sentence:

Read the messy page $\rightarrow$ emit a **typed mention** for each thing it says, with a pointer back to the box it came from $\rightarrow$ decide which mentions are the **same** thing $\rightarrow$ write one **authority record** per thing, listing all its aliases and all its source boxes.

Two vocabulary words carry the whole design.

Mention vs. Entity — the only jargon you must keep straight

A **mention** is one *occurrence* of an entity in the text: the string “Grace.” written in a particular region of a particular page. There are many mentions.

An **entity** is one *real-world thing*: the actual person *Grace Panyard*. There is one entity, gathered from its mentions, with a canonical name and a list of variants.

NER produces mentions; resolution produces entities; the knowledge graph (Part C) is built from entities. Because each entity remembers its mentions' **vertices**, every node in that graph can still point at a box on the scanned manuscript — *provenance-first*, exactly as the research plan demands.

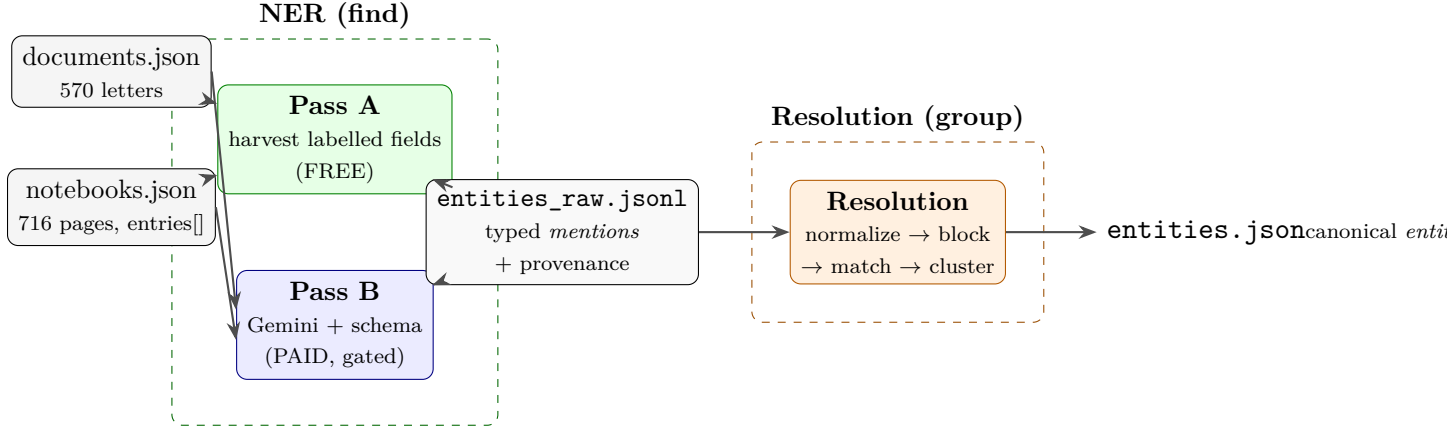


Figure 1: End-to-end. NER *finds* typed mentions (free Pass A + gated paid Pass B), resolution *groups* them into canonical entities. Every arrow preserves provenance so the final entity can still cite a polygon on the page.

2 NER, Pass A: the free structured harvest

step_6 already did real work for us: it split each *letter* into labelled fields and gave each *notebook entry* its own record with a *date*. Those labels *are* type information we paid for once and should never pay for again. Pass A simply reads them off (`harvest_letter`, `harvest_notebook_page`):

step_6 field	mention type	attribute filled
<code>src_recipient</code>	PERSON	role: <code>recipient</code>
<code>src_signature</code>	PERSON	role: <code>sender</code>
<code>src_location_recipient/_sender/src_origin</code>	PLACE	role: <code>..._location</code>
<code>src_date / archv_date</code>	DATE	role: <code>letter_date / archival_date</code>
<code>notebook entry date</code>	DATE	role: <code>entry_date (+page_label)</code>

Two wins, in the words of the code comments: (1) it costs **zero tokens**, and (2) it *removes* the easy entities from the prose, so Pass B can spend its tokens on the genuinely hard stuff. Think of it as picking up the money already lying on the table before paying anyone to look for more.

Each harvested mention is stamped by `_provenance(doc_id, region, page)`, which finds the region whose `category` matches the field — so a recipient mention cites the *recipient's* polygon, not some neighbouring box.

3 NER, Pass B: LLM extraction over the prose

The body of a letter and the text of an entry are where the rich entities hide. Take one real corpus pattern (a notebook favour):

“Marg. Quinn enrolled her neighbor Mr. Maughan, cured of cancer.”

That single clause contains two PERSONS, a RELATION (*neighbor of*), a FAVOR (an *enrollment*) with an inline OUTCOME (*Cured*), and a CONDITION (*cancer*) attached to a *specific* person. A flat extractor loses the bundling; our nested schema keeps it.

3.1 Why a schema, not “return JSON”

Free-form “return JSON” prompts drift: a key gets renamed, a list becomes a string, the model wraps prose around the object. Passing a `types.Schema` switches Gemini into **constrained decoding** — the output is *grammatically forced* to match the shape, so it is always parseable. We pay a small “constrained-decoding reasoning tax,” so the schema gives the model a free-text **reasoning** field *first* (via `property_ordering`): a scratchpad to think in before it commits to the typed lists.

The ontology lives in *one* place (`ENTITY_TYPES + build_response_schema()`) so the schema, the prompt, and any downstream validator can never drift apart. Top-level lists are *nullable*, so an empty page yields empties rather than hallucinated entities; the `SYSTEM_INSTRUCTION` explicitly orders *keep OCR spelling verbatim, never invent, do not infer the year*.

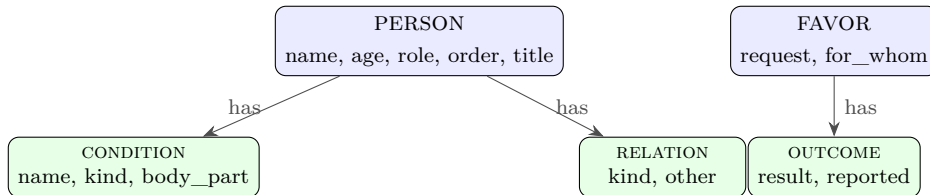


Figure 2: The schema is *nested*: a `CONDITION` and a `RELATION` can hang off the `PERSON` they belong to, and a `FAVOR` carries its `OUTCOME` inline — because in this corpus those facts arrive bundled. After extraction, `_flatten_extraction()` also lifts each nested fact into its own searchable mention, linked back with an `of_person / for_favor` attribute.

3.2 GLiNER vs. the LLM: a two-tier extractor

RESEARCH_PLAN Part B’s ideal is two tiers: a **fine-tuned GLiNER** doing fast, cheap, *local* bulk NER over all $\sim 9k$ chunks, escalating only the hard / low-confidence pages to Gemini. This stage implements the **LLM tier** (the escalation target) end-to-end; GLiNER is the documented cheaper bulk path to add later (custom types, nested spans, proven on historical documents).

	GLiNER (bulk, future)	Gemini structured output (this stage)
cost	local, $\approx \$0$	paid/token (batch -50% , cache prefix)
shines on	high-volume clean spans	nested facts, world knowledge, messy prose
output guarantee	span offsets	constrained-decoded valid JSON
role	first pass on everything	escalation for hard / low-conf pages

3.3 Coreference $\rightarrow$ relation $\rightarrow$ event: the KG trinity

The frontier (LINK-KG, Extract-Define-Canonicalize, Relik) chains three steps: resolve **coreference** (mentions $\rightarrow$ entities, within and across documents), extract **relations** (`WROTE_TO`, `ENROLLED_FOR`, `LOCATED_AT`), then assemble **events** (an enrollment event, an outcome event) with time + provenance. This stage *seeds* all three: the nested schema already captures relations and favour $\rightarrow$ outcome event pairs *within* a passage; the resolver (next section) does the cross-document coreference; full cross-passage relation/event extraction is the Part-C follow-up.

3.4 Cost discipline (nothing bills by accident)

Paid work is deferred behind one switch

By default the stage runs Pass A for the whole corpus, *preview*s Pass B (builds the real prompt + schema, projects the cost), and logs a \$0 projection row so even a dry run leaves a trace in `costs/usage.csv`. Only `execute=True` (`--execute`) spends a cent.

Two levers cut the real bill, both straight from RESEARCH_PLAN Part E:

- **Batch API** (−50%, asynchronous, fine for an offline build). `write_batch_requests()` emits exactly the request file; submission is the deferred paid step.
- **Prompt caching** on the *constant* SYSTEM_INSTRUCTION + `response_schema` prefix — cached reads bill at $\approx 10\%$ of input price. See `build_cache_note()`.

Live runs are **idempotent and resumable**: `_done_unit_ids()` skips any unit already extracted, so a crash at unit 4,000 of 9,000 never re-pays. Every real call routes through `lib.costlog.log(...)`.

4 Entity resolution: the “grouping,” step by step

normalize → block → match → cluster → canonicalize → (authority reconcile)

The running example: the same human appears as "Grace.", "Grace Panyard", "Mrs. Grace Panyard"; OCR turns O'Donnell into ODonnell/O Donnell; a place is Detroit here and Detroit, Mich. there. Resolution decides which of those surface *mentions* point at the same *entity*.

4.1 Step 1 — Normalize: messy surface → stable keys

We never *fix* the text (the original surface is preserved); we only derive *stable comparison keys*. `normalize_tokens()` folds accents, lower-cases, splits punctuation (so O'Donnell, O'Donnell, ODonnell, O Donnell all become the tokens [o, donnell]), strips honorifics (mrs, fr), and expands abbreviations (st→saint, mich→michigan). Then **Double Metaphone** encodes each token by how it *sounds*, so spellings that differ only in silent letters or OCR slips collide (Panyard and Panyord share a code).

Why phonetics, and why *two* keys

A phonetic code maps a word to how it sounds. `phonetic_key` encodes the tokens *one at a time*; `phonetic_key_joined` encodes the tokens *concatenated* (“odonnell”) as one word. We need both because a name one scribe wrote as a single token and another split into two would get different *per-token* keys — but identical *joined* keys — so the joined key is the bridge that keeps those genuinely-equal surfaces in the same bucket. The module **vendors** a compact, dependency-free Double Metaphone so the stage runs with zero installs; `_phonetic()` automatically prefers the `metaphone` / `jellyfish` package if present.

4.2 Step 2 — Block: cheap candidate generation (avoid $O(n^2)$)



Figure 3: Blocking is why resolution is fast. Bucket mentions by a cheap key (`type:phonetic[:location][:year]`) and compare only *within* a bucket. “Grace” never gets compared against “Detroit.” Each mention emits several keys (multi-pass / canopy blocking) and we union the candidates, so a real match survives even if one record lacks a city.

A subtle key (`phj=`) encodes the *joined* tokens, so a name split differently by two scribes still meets in a shared bucket — without it, blocking would propose the pair but the matcher would never see it.

4.3 Step 3 — Match: an ensemble of *interpretable* signals

Within a block, `pair_score()` scores each pair on cheap, interpretable features — interpretable beats a black box here, because David must be able to read the audit log and agree or disagree with each merge.

signal	weight	catches
phonetic equality	0.35	same-sounding OCR / spelling variants
Levenshtein ratio	0.30	single-character slips (panyard vs panyord)
token Jaccard	0.20	reordering / extra tokens
attribute agreement	0.15	same city / role / order where both supply it
embedding cosine	<i>bonus</i>	<i>semantic</i> nearness (gated; blended, not replacing)

$$\text{score} = 0.35 \text{ph} + 0.30 \text{lev} + 0.20 \text{jac} + 0.15 \text{agree},$$

clamped to $[0, 1]$. When the gated embedding signal is present it blends in as $0.85 \cdot \text{score} + 0.15 \cdot \cos$, so the decision stays *mostly* interpretable. Levenshtein is computed on the *spaceless* normalized forms so a stray OCR space doesn’t penalize an otherwise-identical name; attribute agreement is a neutral 0.5 when neither record supplies the attribute.

4.4 Step 4 — Cluster: agreeing pairs $\rightarrow$ entities

Pairwise matches form a graph — mentions are nodes, an edge joins a pair scoring above `MATCH_TAU`. An **entity** is a *connected component*: if $A \sim B$ and $B \sim C$, then A, B, C are one entity (transitive closure). A tiny union-find (`_UnionFind`, path compression + union by rank) computes this in near-linear time, no graph library.

threshold	meaning
<code>MATCH_TAU</code> = 0.62	$\geq$: confident same-entity edge (merge)
<code>AMBIG_TAU</code> = 0.50	$[0.50, 0.62)$: grey zone $\rightarrow$ flag for review / LLM

Any component touched by a grey edge is flagged *ambiguous* and, if enabled, sent to LLM adjudication; otherwise it goes straight to David’s review queue. The thresholds deliberately favour **precision** (see Caveats).

4.5 Step 5 — Canonicalize: one authority record per entity

`canonicalize()` turns each component into one record: pick the fullest, best-attested surface as `canonical_name` (“Grace Panyard” beats “Grace.” — more tokens, tie-broken by frequency), collect every distinct surface as `variants[]`, majority-vote the attributes, gather every mention’s provenance, and compute a `confidence` that blends internal name agreement with the OCR `min_conf` of the source regions. `needs_review` marks the grey clusters.

Listing 1: A canonical record (`data/entities.json`).

```
{
  "id": "person:grace_panyard:0007",
  "type": "PERSON",
  "canonical_name": "Grace Panyard",
  "variants": ["Grace.", "Grace Panyard", "Mrs. Grace Panyard"],
  "attrs": {"city": "Detroit"},
  "mentions": [
    {"mention_id": "...", "surface": "Grace.",
      "provenance": {"doc_id": "...", "rid": "...", "page": 3,
        "vertices": [[x,y],[x,y],[x,y],[x,y]], "min_conf": 0.948}}
  ],
  "confidence": 0.91,
  "needs_review": false,
  "authority": {"wikidata": null, "viaf": null, "geonames": null, "getty_tgn": null}
}
```

5 The three gated power-ups

Each is a real function with a `gated=True` guard, a clear TODO, and cost-logging — wired and documented, but **never run by default**.

1. **Embedding similarity** (`embedding_signal`). Cosine over mention embeddings, computed *only* for the pairs blocking already proposed (never full n^2). Phonetics are blind to *meaning*: “the Capuchin friary at Harlem” and “St. Bonaventure Monastery” share no characters but embed near each other. Routes through `lib.providers.embed`; the *free local* path (`bge-small`, \$0) is the default, paid spaces are key-gated.
2. **LLM adjudication** (`llm_adjudicate`). Only on clusters flagged ambiguous, the model returns a *structured* keep/split verdict (auditable, not prose). This is the 2025-SOTA tiebreaker (in-context clustering / LLM-as-judge, ~94% on name variants). A `split` verdict is *recorded in the audit, not auto-applied* — David reviews before any re-partition.
3. **Authority reconciliation** (`reconcile_person`, `reconcile_place`). Link persons → **Wikidata/VIAF**, places → **GeoNames/Getty TGN**, for global IDs, disambiguation, enrichment, and Linked-Data export. These public APIs don’t bill, but every call is still cost-logged at \$0 so the ledger is complete.

6 Scale-up path: Splink

For a *much* larger corpus, `splink_plan()` documents the drop-in swap to **Splink** (probabilistic record linkage; Fellegi–Sunter with EM-learned match weights; DuckDB/Spark; millions of records

on a laptop). Our pieces map almost one-to-one, so this is a scale path, not a rewrite:

this module	Splink
<code>blocking_keys()</code>	<code>blocking_rules_to_generate_predictions</code>
<code>pair_score()</code> signals	<code>comparisons / ComparisonLevels</code> (jaro_winkler, exact, custom-sql)
<code>cluster_mentions()</code>	<code>cluster_pairwise_predictions_at_threshold</code>

Precompute the `phonetic_key / norm_name` columns with this module's functions and feed them to Splink, so blocking and comparisons reuse exactly the same normalization.

7 How to run

```
# from step_7/ with the venv active

# NER -- free Pass A + a no-spend preview of Pass B
python stages/extract_entities.py --limit 25
python stages/extract_entities.py --batch # emit Gemini Batch file (no spend)
python stages/extract_entities.py --execute # PAID live LLM pass (authorize first)

# Resolution -- offline rule-based core ($0)
python stages/resolve_entities.py
python stages/resolve_entities.py --use-embeddings # free local bge by default
python stages/resolve_entities.py --use-llm # GATED; bills tokens
python stages/resolve_entities.py --reconcile # Wikidata/VIAF/GeoNames/Getty ($0)
```

Artifacts (all new files — non-destructive): `data/entities_raw.jsonl` (mentions), `data/entities.json` (canonical records + `_meta`), `data/entities_merge_audit.jsonl` (one row per merge decision), `data/entities_batch_requests.jsonl` (optional batch file).

8 Caveats

Read before trusting a run at scale

- **A wrong merge is worse than a wrong split.** Merging two real people pollutes every query about either; a split merely scatters one person across two records. Thresholds therefore favour precision, and the grey zone routes to review. Tune `MATCH_TAU` / `AMBIG_TAU` on David's gold first.
- **Mention-schema seam between the two stages.** `extract_entities.py` currently emits mentions keyed `text` / `unit_id` / `pass`, while `resolve_entities.py` reads `surface` / `mention_id` (its `SCHEMA` block documents the expected contract). Until a tiny adapter bridges them, the resolver runs on its *demo fallback* (recipient names from the letters' structured fields), clearly labelled in the output `_meta.source`. A known, small follow-up — flagged so the next run isn't surprised.
- **Pass B is deferred.** No live extraction has run; Pass-B figures are projections. Schema, prompt, batch file, and resume logic are real and dry-run tested.
- **The vendored Double Metaphone is a pragmatic subset** (high-value rules, not Philips' full ~250-line algorithm) — good enough for blocking; install `metaphone/jellyfish` for the full version.
- **The LLM split verdict isn't auto-applied** yet; it is recorded for review. Authority lookups are best-effort and never crash a run.
- **Prices are placeholders.** Confirm provider pricing before any billed run; nothing bills until a stage is explicitly executed.

9 Sources

- **GLiNER** (custom + nested NER): <https://github.com/urchade/GLiNER>; fine-tuning generalist NER: <https://labelstud.io/blog/fine-tuning-generalist-models-for-named-entity-recognition>; historical NER (Zibaldone): <https://arxiv.org/pdf/2505.20113>.
- **Structured / constrained output:** Outlines on AWS <https://aws.amazon.com/blogs/machine-learning/generate-structured-output-from-llms-with-dottxt-outlines-in-aws/>; JSONSchemaBench <https://arxiv.org/pdf/2501.10868>.
- **Coref → relation → event → KG:** LINK-KG <https://arxiv.org/html/2510.26486>; Extract-Define-Canonicalize <https://arxiv.org/pdf/2404.03868>; Relik <https://neo4j.com/blog/developer/entity-linking-relationship-extraction-relik-llamaindex/>.
- **Entity resolution:** Splink <https://github.com/moj-analytical-services/splink>; ER intro <https://medium.com/@adev94/entity-resolution-an-introduction-fb2394d9a04e>; LLM in-context clustering ER <https://arxiv.org/abs/2506.02509>.
- **Authority control:** Wikidata, VIAF, GeoNames, Getty TGN public reconciliation endpoints.
- **Cost:** prompt caching 2026 <https://www.digitalapplied.com/blog/prompt-caching-2026-cut-llm-costs>; Gemini pricing <https://costbench.com/software/llm-api-providers/google-gemini-api/>.